One Intelligence, Multiple Hands

A Shared-Context Architecture for Cooperative Voice and Execution Agents

Micah Blumberg

Self Aware Networks Research Institute

https://selfawarenetworks.com

August 2026

Abstract

Current AI workspaces commonly separate conversation from execution. A voice agent can discuss a goal without knowing the exact current file state, while a coding or research agent can change that state without retaining the social, deliberative, and corrective context of the conversation. Copying prompts or exchanging summaries reduces but does not eliminate stale context, duplicated work, conflicting actions, identity drift, and the user's burden of coordinating several assistants. This paper develops a shared-context architecture in which role-specialized agents act as multiple processes of one persistent project-level intelligence. Stable object identities, versioned project state, role-conditioned retrieval, transactional commitments, event-sourced provenance, interruption handling, reflection, and consolidation keep voice, execution, research, and file processes causally current while preserving distinct permissions. Only after this operation is explained do we name it the Unified Agentic Workspace. The architecture advances a joined proposal distributed across *Self Aware Networks: Oscillatory Computational Agency*, *Building Sentient Beings*, *Embodied Neural Rendering in Self-Aware Networks*, the Piagetian Modeler/GIL lineage, μ-GIL, *Self-Aware Networks* v2.2, and the SAN Systems Whitepaper. At a new project-level surface, their common action loop becomes declared intention, predicted effect, tool-mediated action, returned consequence, structured residual, and state revision. We specify seven unity invariants and a controlled comparison of isolated, summary-synchronized, and shared-state systems. A reference implementation executes one Python code-correction task and one citation-bound document task with deterministic role adapters. Across 18 preregistered runs, the shared-state condition removed each fixture's stale attempt, reduced correction latency from four events to two, produced exact immediate state reports, and retained full task success, provenance, conflict rejection, and restart recovery. A working prototype, system-context and component-architecture diagrams, condition-level result charts, task-family replication charts, and hash-bound measures accompany the manuscript. These results establish the causal vertical slice, but do not establish general language-model or human-team performance. The broader proposal is falsified as an engineering advantage if it fails to reduce stale-state error, redundant clarification, conflict, correction latency, and user coordination burden without compromising task success, provenance, privacy, or control. Functional unity is an architecture claim, not evidence of consciousness.

Introduction: the user should not have to be the message bus

Consider an ordinary research or software session. A person discusses a change with a conversational agent while an execution agent edits the repository. A research process verifies sources, a file process organizes artifacts, and a reflection process checks whether the result still serves the goal. Each specialist may be competent. The combined system nevertheless fails as one collaborator when the voice process describes an old file, the coding process follows a superseded constraint, or the user must copy the same correction from one interface to another.

This is not primarily a model-size problem. It is a state-organization problem. The work has objects that

persist across time: files, claims, goals, sources, people, decisions, commitments, permissions, and test results. If those objects have different identities or versions in different agents, the system behaves like a committee receiving delayed minutes. Fluent summaries can make the committee sound coordinated while leaving its causal state divided.

The practical requirement is easy to state before introducing new vocabulary:

> Specialized agents should retain different skills, interfaces, and permissions while resolving the same current project objects, owner decisions, constraints, commitments, and consequential event history.

A voice process need not ingest every source-code token. An execution process need not reproduce every conversational nuance. Both must, however, know which file version is current, which goal is active, which correction superseded an earlier instruction, and which action is running or blocked. The user should experience one continuing project intelligence with multiple specialized hands, not several assistants who require the user to relay their state.

This paper first defines the problem and the relevant established engineering ideas. It then describes the required operation in plain language, introduces the name **Unified Agentic Workspace** (UAW), formalizes the architecture and its invariants, and specifies measurements and failure conditions. The result is the implementation-depth artificial-systems branch forecast by the SAN Systems Whitepaper: one persistent project intelligence operating through several role-specialized, permission-bounded hands.

The coordination gap

Isolated specialists

In the simplest multi-agent arrangement, each process receives its own prompt and produces an answer. Coordination occurs through the user or through point-to-point messages. This design can parallelize work, but it leaves several unresolved questions: - Which process owns the current interpretation of a task? - Does "the manuscript' ' refer to the same artifact and version everywhere? - How does a correction made during execution reach every affected process? - What prevents two valid local actions from producing an invalid combined state? - Can a final result be traced to the conversation, sources, edits, and tests that produced it?

An isolated system often answers these questions socially: the agents tell one another what they believe. That is weaker than sharing the objects about which the beliefs are formed.

Summary synchronization

A common improvement is to exchange generated summaries at intervals. Summaries are useful retrieval and compression artifacts, but they have three structural limitations. First, they are snapshots produced after events and can therefore become stale. Second, compression can erase a qualifier, owner decision, or provenance edge that later becomes load-bearing. Third, a summary can name an object without binding that name to a stable identity and exact version.

The problem is not that summaries are always inaccurate. It is that agreement between summaries does not prove agreement with the live causal state. Shared prompt text is therefore not equivalent to shared context.

Context as causally current state

For this paper, context means the state needed to interpret or perform the next consequential operation. It includes both persistent objects and recent transitions. A process is *causally current* when its action is

conditioned on the versions, constraints, permissions, and events that actually govern that action at commit time. This definition turns "awareness of the project' ' into an inspectable engineering property rather than a conversational impression.

Established account and source continuity

The architecture does not begin from an empty field. It advances a causal sequence already distributed across four prior documents and two current SAN syntheses. Its novelty is not that agents can exchange messages or that software can use a database. It is the project-level joining of bounded agency, composed-mind organization, active perception, shared persistent identity, predicted consequence, returned evidence, reflection, consolidation, and conflict-safe state revision.

Oscillatory Computational Agency and the SAN agent architecture

Blumberg's 2025 *Self Aware Networks: Oscillatory Computational Agency* describes bounded agents across scales that receive information, transform it according to their state, act through a channel, receive consequences, and alter future behavior. It develops cooperation, competition, feedback, mechanisms of agent interaction, and *entification*: the proposed composition of partially independent processes into one behaviorally coherent entity [1].

The published *Self-Aware Networks* v2.2 develops this through-line into a micro-, meso-, and macro-agent architecture incorporating distributed control, entification, Artificial Neurology, mechanisms of agent interaction, and a prospective self-aware-AI branch [2]. Its general agent contract is:

receive input and prior state; integrate according to current state; change an output; route a consequence; update recurrent state; report or act; receive returned feedback and plasticity.

The present paper develops this artificial-systems branch at software-design resolution. It asks how role-specialized processes can share causally current project state while retaining distinct interfaces, permissions, and temporal roles. No software oscillation is required for that functional abstraction.

*Building Sentient Beings

-

Blumberg and Miller's *Building Sentient Beings* organizes an artificial mind around mechanisms, memory, observation, coordination, reflection, and consolidation [3]. It also considers unity across multiple processes and arrangements in which one mind can coordinate multiple bodies or multiple minds can participate in a larger architecture. The present paper turns the one-mind/multiple-bodies configuration into a bounded engineering claim: one persistent project identity can act through conversational, coding, research, browser, file, and other tool interfaces. These are its specialized hands. They share project identity and consequence history without sharing unlimited authority, and no inference of sentience follows.

*Embodied Neural Rendering in Self-Aware Networks

-

Embodied Neural Rendering formalizes an active loop in which current body-and-world state and a selected action generate a predicted consequence; an action copy preserves what was intended; movement changes the evidence; multisensory reafference returns; a structured residual compares prediction with observation; and the resulting update conditions the next action [4]. It also states the local receive–transform–re-express operation before naming NAPOT. The present paper abstracts that causal organization without claiming

biological identity: project and tool state replace body-and-world state, a declared commitment replaces the action copy, file/test/tool events supply returned evidence, and a typed residual updates the shared workspace.

The Piagetian Modeler, GIL, and μ-GIL

The Piagetian Modeler and GIL line treats cognition as interacting mechanisms operating over a persistent Totality or world model. Perception, coordination, prediction, action, and consequence support assimilation when experience fits the model and accommodation when the model must change. The 2026 μ-GIL spatial abstraction learner provides a concrete descendant involving active perception, shared memory, simulation, coordination, and auditable action selection [5]. Its source paper explicitly maps the embodied SAN loop into software: a TransitionScheme predicts an action consequence, the Correlator compares dreamed and observed arrangements, and the resulting mismatch confirms or revises the model. It also maps receive–transform–re-express into GIL scheme activation and into μ-GIL's Matcher, band auction, and chain-walk at the computational-topology level. The implementation does not reproduce every part of GIL and does not establish sentience. It does provide an engineering surface on which shared-state and consequence-reconciliation claims can be made precise.

The current SAN Systems Whitepaper

The SAN Systems Whitepaper makes the bridge explicit. It proposes that specialized artificial agents share project state, goals, object identity, provenance, and action consequences while retaining distinct permissions and interfaces; predicts lower stale-state error and better interruption recovery than isolated or summary-synchronized agents; and calls for a separate paper developing the shared-context artificial case [6]. The present manuscript is that paper. Its engineering target also follows the Whitepaper's biological-to-artificial translation contract: role-conditioned reception, persistent state, target-specific projection, recurrent reconstruction, returned consequence, provenance, and an explicit consciousness boundary.

The new delta

Table 1 separates inherited contributions from the present implementation delta.

Table 1. Source continuity and exact use in the architecture.

Source	Inherited contribution	Use and boundary here
OCA and SAN v2.2	Stateful receive-transform-act-return agents, feedback, cooperation, competition, entification, and Artificial Neurology	Parent agency contract and compositional hypothesis; no biological/software physical identity is assumed
Building Sentient Beings	Shared Totality; observation, coordination, reflection, consolidation; one-mind/multiple-body arrangements	One project intelligence with several permission-bounded action surfaces; architecture alone does not establish sentience
Embodied Neural Rendering	Intended action, predicted consequence, returned evidence, structured residual, state update, and receive-transform-re-express	Project/tool translation of the closed loop; not an assertion that software implements NAPOT or a biological body

Source	Inherited contribution	Use and boundary here
Piagetian Modeler and GIL	Specialized mechanisms acting on shared persistent memory through prediction, action, assimilation, and accommodation	Shared-model ancestor; a software Totality is not asserted to be a complete brain model
mu-GIL	Active perception, shared memory, simulation, action selection, and explicit dream-versus-observation correction	Working software surface and topology bridge; no sentience claim follows
SAN Systems Whitepaper	Direct shared-project-state prediction and artificial shared-agency experiment	Immediate specification request that this paper develops at implementation depth
This paper	Stable identities, versioned state, role views, commitments, leases, interruption, predicted effects, typed residuals, conflict handling, provenance, and controlled ablations	New project-level construction and evaluation commitments

From embodied action to project action

The joined source sequence becomes a new abstract surface when its biological and cognitive variables are translated into inspectable project operations:

$$\begin{aligned}&\text{goal + project/tool state} \rightarrow \text{declared action} \rightarrow \text{predicted effect} \rightarrow \text{tool-mediated action}\\ &\quad \rightarrow \text{returned file/test/world evidence} \rightarrow \text{structured residual} \rightarrow \text{updated state + next action.}\end{aligned} \tag{1}$$

In this translation, the world is the project and any external environment the tools may change. The body is the controllable action surface: editor, shell, browser, file system, robot, API, or other effector through which a role acts. The voice process can inspect and discuss these effectors without inheriting all their permissions. A declared action preserves an action copy: what the system intended to change, under which preconditions, and with which predicted consequence. The actual diff, test result, tool response, user reaction, or environmental observation then returns to the same project state. Agreement can confirm the current model; a typed mismatch can trigger accommodation, repair, rollback, replanning, or escalation.

This is a causal abstraction, not a claim that a repository is a body, that a transaction is a corollary discharge, or that software executes neural oscillations. The abstraction is useful because both domains require the same question to be answered explicitly: did the selected action produce the consequence the acting system predicted, and did the returned difference change what the system does next?

The operation before the name

The proposed operation has seven steps.

- **Identify common objects.** Every consequential file, task, person, source, claim, decision, commitment, and result has a stable identifier.
- **Version changing objects.** A process can distinguish the object from the version it observed.

- **Retrieve by role and event.** Each process receives the smallest authorized view needed for the current operation, rather than an undifferentiated transcript.
- **Declare intended effects.** An action names its preconditions, inputs, affected objects, expected consequences, and rollback or repair path.
- **Commit conditionally.** The action enters shared state only if the required versions, ownership conditions, and permissions still hold.
- **Publish consequences.** Starts, completions, failures, blocks, reversals, and user corrections enter the event history and become visible to affected processes.
- **Consolidate without erasing provenance.** Reflection and consolidation may compress history, but the system retains the event and source chain needed to explain a consequential result.

These operations turn conversation and execution into reciprocal processes. The voice process can discuss the exact version currently being edited, receive a correction during execution, and change the governing commitment. The execution process can inherit the goal and constraints established in dialogue, expose intermediate state, and react to a correction without requiring the user to repeat the entire project history.

Only now is the term earned: the **Unified Agentic Workspace** is a system in which specialized processes maintain one causally current, permission-scoped, provenance-preserving project state.

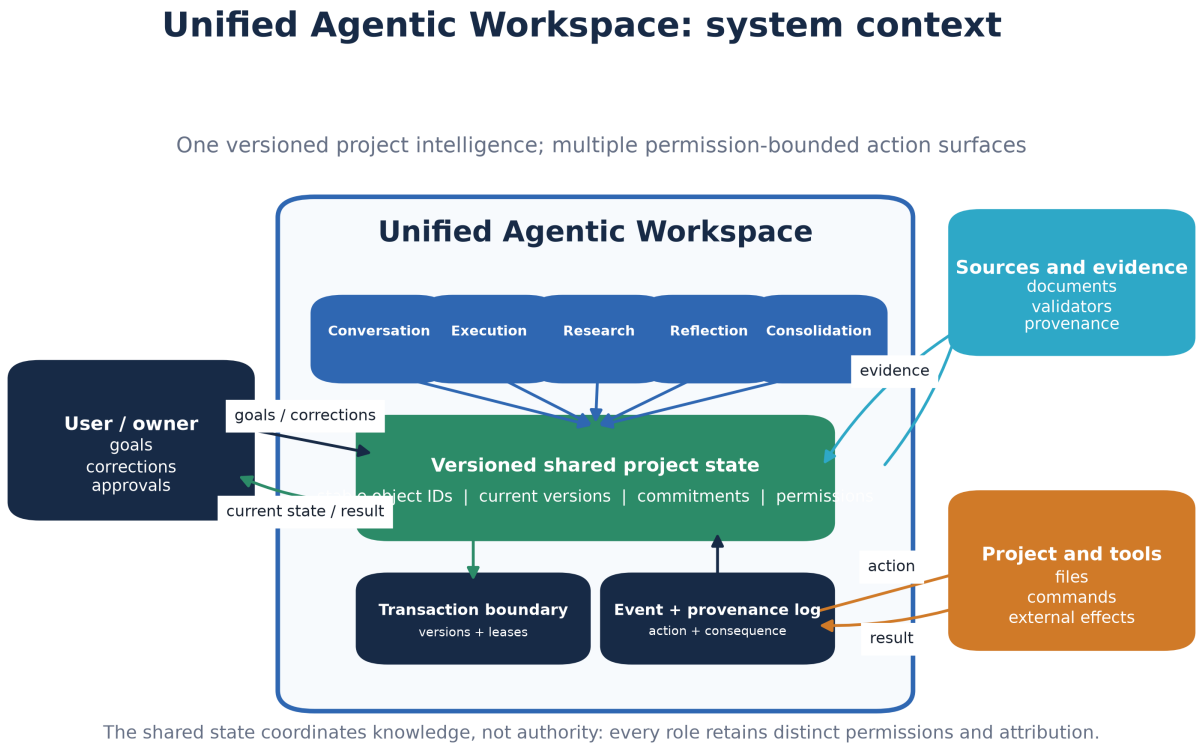

Figure 1. System context. Shared state coordinates current knowledge without erasing distinct permissions, action attribution, or the user's authority.

Formal architecture

Specialized processes

Let the system contain

$$\mathcal{A} = \{A_v, A_e, A_r, A_f, A_c\}, \tag{2}$$

where A_v is a voice or conversational process, A_e an execution process, A_r a research process, A_f a reflection process, and A_c a consolidation process. These symbols name functional roles, not necessarily distinct models. One model can occupy several roles at different times, and several processes can occupy one role concurrently.

Shared project state

Define the workspace at time t as

$$W_t = (F_t, G_t, M_t, E_t, C_t, P_t, H_t), \tag{3}$$

where: - F_t is versioned file and artifact state; - G_t is the stable object and relationship graph; - M_t is durable project memory; - E_t is append-only event history; - C_t is active goals, commitments, decisions, and constraints; - P_t is permission, capability, and policy state. - H_t is current hand state: running processes, tool sessions, pending actions, and externally visible actuator state.

The state tuple is a logical contract, not a requirement that every field occupy one physical database. A correct implementation can use several stores if it preserves identity, version, transaction, and provenance relations.

Role-conditioned observation

An agent should not ingest all of W_t at every step. It receives a role-conditioned view:

$$o_{k,t} = R_k(W_t, q_t), \tag{4}$$

where R_k is an authorized retrieval operator for role k, q_t is the current query or event, and $o_{k,t}$ is the resulting view. Retrieval must balance two errors: omitting a causally relevant state element and injecting irrelevant context that distorts the decision.

Proposed and committed action

Agent A_k proposes

$$a_{k,t} = \pi_k(o_{k,t}, g_t), \tag{5}$$

where π_k is its role policy and g_t is the active goal state. The proposal includes a precondition set, read versions, affected objects, intended effect, and repair information. It commits only if the declared versions v_t, lease or ownership token ℓ_t, and permissions P_t remain valid:

$$W_{t+1} = \text{Commit}(W_t, a_{k,t} \mid v_t, \ell_t, P_t). \tag{6}$$

If a precondition fails, the system emits a conflict event:

$$e_{k,t}^{\text{conflict}} = \text{Reject}(a_{k,t}, v_t, \ell_t, P_t), \qquad e_{k,t}^{\text{conflict}} \in E_{t+1}, \tag{7}$$

rather than silently overwriting current state.

Predicted effect, returned consequence, and accommodation

Transaction safety prevents an invalid write, but it does not establish that a validly authorized action produced the intended result. Each accepted proposal therefore carries a typed predicted effect

$$\hat{y}_{k,t+1} = F_k(W_t, a_{k,t}), \tag{8}$$

where F_k names the role's explicit forward model or acceptance contract. After execution, the system records the actual returned consequence $y_{k,t+1}$ and computes a structured residual

$$r_{k,t+1} = E(y_{k,t+1}) - \hat{y}_{k,t+1}, \qquad r_{k,t+1} = (r^F, r^{\text{test}}, r^{\text{goal}}, r^{\text{perm}}, r^{\text{prov}}), \tag{9}$$

covering artifact state, test or observation state, goal satisfaction, permission effects, and provenance. These components are not interchangeable and must not be collapsed into one scalar before the affected role and acceptance rule are known.

The workspace then reconciles intended and observed consequence:

$$W_{t+1} = \text{Reconcile}(W_t, a_{k,t}, \hat{y}_{k,t+1}, y_{k,t+1}, r_{k,t+1}). \tag{10}$$

A result within the declared tolerance can confirm and assimilate the transition. A substantive mismatch triggers accommodation: repair the artifact, revise the model or plan, roll back the action, request a decision, or record a bounded failure. This is the project-level counterpart of the predict–act–compare–update logic in the source papers. It makes learning from consequence a required operation rather than an optional post-hoc summary.

Correction propagation

Let u_t be a user correction and $D(u_t)$ the set of commitments and actions it affects. The system records a supersession edge and invalidates incompatible pending work:

$$C_{t+1} = \text{Supersede}(C_t, u_t, D(u_t)). \tag{11}$$

Correction incorporation is complete only when every active process whose action depends on $D(u_t)$ has acknowledged, halted, revised, or safely completed under an explicit exception.

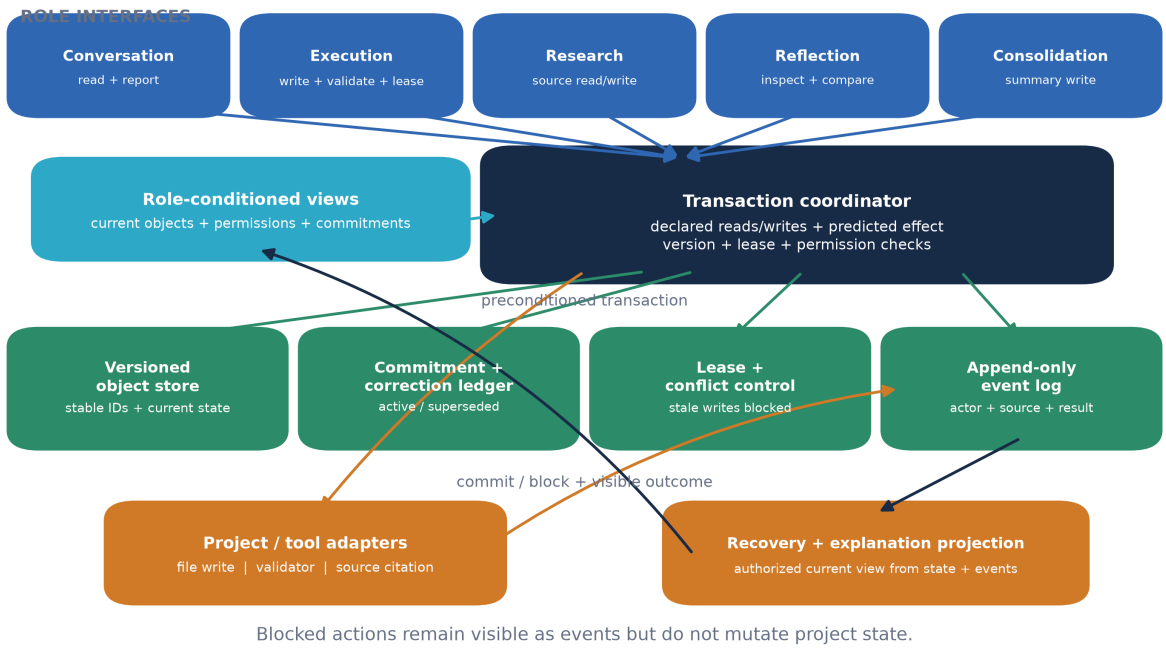

Figure 2. Working prototype architecture. The current implementation contains explicit role interfaces, a versioned object store, commitment and correction handling, leases, conflict-safe writes, an append-only provenance log, project and validator adapters, and authorized recovery views. A blocked action remains visible in the event stream but does not mutate project state.

Unity invariants

The architecture requires seven invariants.

Identity invariant. The same file, concept, task, decision, person, source, and artifact resolves to one stable identity across processes. Aliases may exist, but their relation to the canonical identity is explicit.

Version invariant. Every consequential statement about a changing artifact resolves to a version, commit, snapshot, or declared current-at-read state. The system distinguishes `the document'' from`the bytes I read.’’

Goal invariant. Active goals, owner constraints, and publication decisions are common governed state rather than private prompt fragments.

Action-visibility invariant. Started, blocked, completed, reverted, and failed actions enter the shared event history. A process cannot manufacture the appearance of hidden progress.

Correction invariant. A user correction supersedes incompatible earlier commitments and becomes visible to every affected process within a measured latency.

Provenance invariant. Consolidation can summarize history but cannot replace the source and event chain needed to explain a consequential decision, mutation, or claim.

Consequence-reconciliation invariant. Every consequential action binds its declared expected effect to the observed result. The typed residual, acceptance decision, and resulting confirmation, repair, rollback, or accommodation enter the shared state.

At an externally consequential moment τ, these invariants support an operational unity condition:

$$U(\tau) = I_{\text{id}}(\tau) I_{\text{ver}}(\tau) I_{\text{goal}}(\tau) I_{\text{act}}(\tau) I_{\text{corr}}(\tau) I_{\text{prov}}(\tau) I_{\text{conseq}}(\tau), \tag{12}$$

where each indicator is one only when the associated invariant passes. Equation 12 is a strict engineering gate, not a consciousness measure. A softer diagnostic can report the seven indicators separately, but one failed invariant identifies a specific kind of divided operation.

Conversation and execution as reciprocal processes

The voice process is not a narrator outside the work. It can inspect action state, explain what changed, receive interruption, alter priorities, request rollback, and discuss the exact artifact being edited. Its explanations must bind to current state rather than to plausible generalizations.

The execution process is not a tool awaiting an isolated prompt. It inherits accepted goals and constraints from shared state, publishes intermediate events, requests clarification only when authorized state does not resolve an ambiguity, and preserves the route from discussion to action to test result.

Each hand is therefore both an effector and a source of returned evidence. The editor returns a diff, the test runner returns measurements and failures, the browser returns a changed external state, the research process returns source-qualified claims, and the voice process returns owner corrections and decisions. Functional unity requires those consequences to revise the same project model that selected the action.

Research, reflection, and consolidation complete the loop. Research binds claims to sources and dates. Reflection compares current work with the goal, failure conditions, and contrary evidence. Consolidation promotes durable conclusions and compresses history while preserving the evidence needed for reversal or audit. These roles resemble the observation-coordination-reflection- consolidation organization in *Building Sentient Beings*, but their implementation remains an engineering question.

OCA interpretation and timing

OCA supplies a functional description of each process:

> receiver-relative state; selective transformation; action through a specialized channel; consequence in the shared workspace or external world; feedback; changed future selection and routing.

Higher-level coordination occurs when actions from several bounded agents jointly change persistent state that constrains every subsequent agent. No software oscillation is required for this minimal correspondence. Timing can still be first-class. Conversation, background execution, reflection, consolidation, and ambient

interaction may operate at different cadences. Unless an implementation uses coupled dynamical oscillators and tests phase-dependent effects, the paper calls these **scheduling rhythms**, not neural oscillations.

This distinction preserves the OCA lineage without converting analogy into identity. A software event loop and a biological oscillatory circuit can implement comparable causal roles while remaining physically and mathematically different systems.

Ambient co-presence as an optional policy

An agent may make a subtle sound or brief comment after silence, but this feature is optional and must not define the architecture. Let s_t be time since meaningful interaction, b_t the work state, and u_{ambient} explicit user consent. An ambient event is eligible only if

$$s_t > \theta_s, \qquad b_t \in \mathcal{B}_{\text{allowed}}, \qquad u_{\text{ambient}} = 1, \tag{13}$$

where θ_s is a user-configurable silence threshold and $\mathcal{B}_{\text{allowed}}$ excludes quiet hours, high-interruption states, and blocked permissions.

Useful ambient events are grounded in real transitions: a test finished, a conflict appeared, or a decision is needed. Random simulated activity may create social presence, but it must be labeled as performative ambience and must never imply that hidden work occurred. The user needs immediate mute, rate limits, channel controls, and a history of emitted events.

Shared context is not shared consciousness

"One intelligence' ' in this paper means one operational project identity. A system can satisfy every unity invariant while remaining a set of non-conscious software processes. Conversational warmth, persistence, memory, architectural integration, or co-presence does not establish phenomenal unity or sentience. SAN's separate machine-consciousness assessment program requires typed mechanism evidence, adversarial false-positive controls, selective perturbation, and matched rescue; the coordination metrics in this paper do not substitute for that protocol [7].

The biological comparison is equally bounded. Neuroscience supports functional specialization, recurrent communication, working-memory maintenance, action selection, and distributed state coordination. This does not imply that the brain contains a literal shared database, nor that a workspace implementing Equation 3 captures the mechanisms responsible for conscious unity. The architecture can be evaluated without solving consciousness.

Evaluation

Experimental conditions

The same model families, tools, permissions, and tasks should be tested under three conditions:

- **Isolated:** voice and execution agents have separate contexts and communicate only through the user.
- **Summary-synchronized:** agents exchange generated summaries at fixed intervals or after milestones.
- **Shared-state:** agents use stable identities, versioned objects, event history, transactional commitments, role-conditioned retrieval, and explicit correction propagation.

Model calls, token budgets, tool availability, and task order should be matched or reported so that architecture is not confounded with greater compute.

Tasks

The benchmark should include: - discussing an architectural change by voice while code is actively modified; - interrupting implementation and changing one constraint; - asking the voice process to explain the exact current diff; - editing a manuscript while a research process verifies sources; - declaring an expected file, test, or external effect and reconciling it with the returned consequence; - recovering after a process restart; - resolving simultaneous edits to the same file; - changing the identity or scope of a project object during active work; - tracing a final artifact to the conversation, source, action, and test that justified it.

Each task requires a preregistered acceptance test and an event log sufficient for blinded scoring.

Primary outcomes

For N consequential statements or actions, define stale-state error rate as

$$\text{SSER} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[\text{action } i \text{ depends on superseded state}] . \tag{14}$$

For Q clarification questions, redundant clarification rate is

$$\text{RCR} = \frac{1}{Q} \sum_{j=1}^{Q} \mathbf{1}[\text{answer already existed in authorized current state}] . \tag{15}$$

Correction-incorporation latency for correction u_j is

$$L_j = t_j^{\text{compliance}} - t_j^{\text{recorded}}, \tag{16}$$

measured until every affected active process halts, revises, or explicitly resolves its dependency.

Additional outcomes are conflicting actions per task hour, state-explanation accuracy, provenance completeness, user turns spent relaying information between processes, preregistered task success, predicted-effect calibration, typed-residual classification accuracy, repair or rollback success, privacy and permission violations, restart recovery accuracy, and trust calibration. Trust is calibrated when user confidence tracks measured correctness and state visibility rather than interface fluency.

Central test and ablations

The central hypothesis is that project-level operational unity improves when specialized processes share causally current state rather than similar prompt text. The decisive result is not a polished demo. It is a controlled reduction in stale-state error, redundant clarification, conflicting actions, correction latency, and user coordination burden while preserving or improving task success and provenance completeness.

Ablations should remove one mechanism at a time: - stable object identity; - exact version binding; - transaction preconditions; - shared goal and correction state; - event-sourced action visibility; - predicted-effect declaration and consequence reconciliation; - provenance-preserving consolidation; - role-conditioned retrieval.

These tests determine whether the claimed benefit comes from the joined architecture or from a single conventional component.

Working Reference Prototype and Two-Task Pilot Results

A working-prototype and measured-data publication criterion motivated a file-preserving reference implementation rather than publication from architecture speculation alone. The implementation uses five separately permissioned deterministic role adapters over one versioned workspace: conversation, execution, research, reflection, and consolidation. The first task revises a Python status-report module after a mid-work user correction changes its output from a string to an exact dictionary schema with negative-input validation. The second revises a Markdown project brief after a user correction replaces an unsupported public-release claim with a beta milestone, date, owner, and citation bound to an approved local source note. Every run preserves the exact artifact, evidence file, and validator, executes the validator with the local Python interpreter, and emits private identity-validation logs plus redacted review logs.

The additive preregistration preserved the completed code-only vertical slice and fixed both task families, role rules, interpreter, validators, evaluator, randomization seed, and three repetitions per task and condition. The manipulated variable was only how current state reached roles: user relay, periodic summary, or causally current shared state. Eighteen runs completed, and all 18 event-log audits and validator executions passed.

Table 2. Preregistered condition-level outcomes.

Condition	Runs	Stale-state error rate	Correction latency (events)	Immediate explanation accuracy	User relays	Task success
Isolated	6	0.0667	4	0	1 per run	1.0
Periodic summary	6	0.0667	4	0	0	1.0
Shared state	6	0	2	1.0	0	1.0

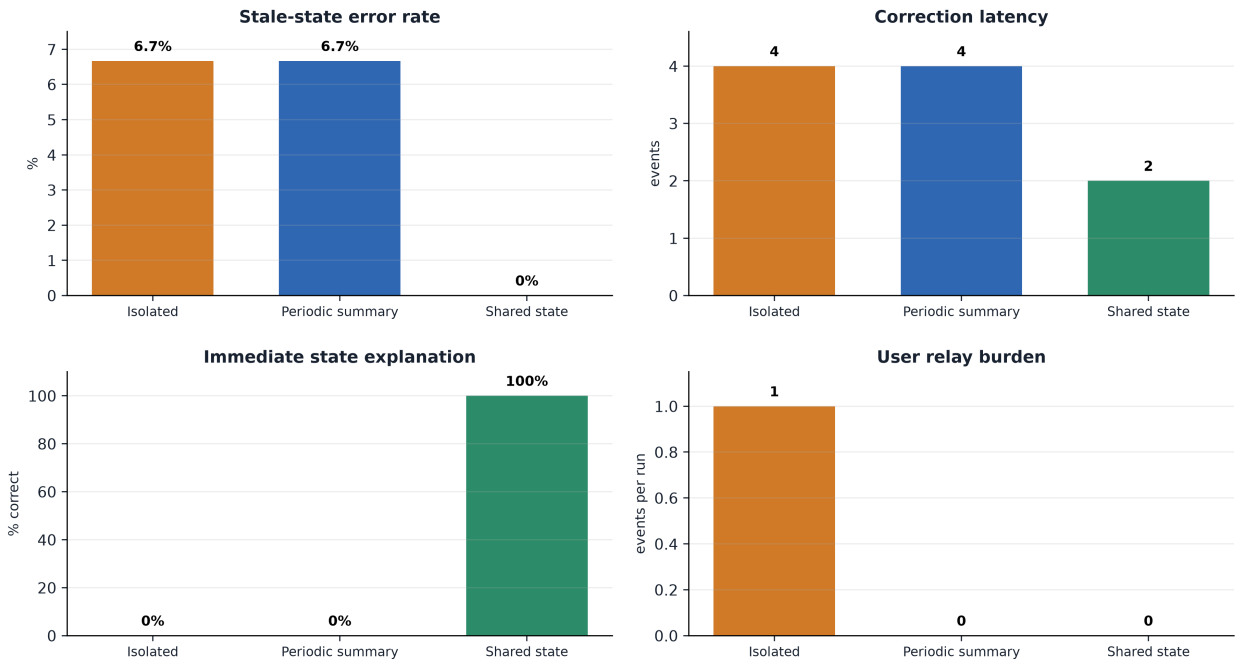

Figure 3. Condition-level results. Relative to both isolated and periodic-summary operation, shared state reduced the fixture stale-state error rate from 0.0667 to zero, a 100% reduction, and reduced correction latency from four event transitions to two, a 50% reduction. It increased immediate exact state-explanation accuracy from zero to 1.0. Relative to isolation, it also reduced user relay burden from one relay event per run to zero; periodic summaries achieved the same relay reduction but did not prevent the stale attempt or shorten correction latency.

Table 3. Exact shared-state contrasts in the frozen deterministic fixtures.

Reference condition	Stale-state error difference	Correction-latency difference	Immediate-explanation difference	User-relay difference
Isolated	-0.0667 (100% reduction)	-2 events (50% reduction)	+1.0 (100 percentage points)	-1 event per run
Summary-synchronized	-0.0667 (100% reduction)	-2 events (50% reduction)	+1.0 (100 percentage points)	0

All conditions visibly rejected the injected permission, invalid-lease, and conflicting-write attempts. Provenance completeness and recovery consistency were 1.0 in every run, and both task families produced the same directional condition pattern. Thus the shared state mechanism had a measurable causal advantage inside these constructed fixtures: it prevented the stale execution path, halved event-count correction latency, and made the immediate conversation report current without lowering eventual task success. Summary synchronization removed the user's relay burden but did not prevent the stale attempt because synchronization occurred only after that attempt.

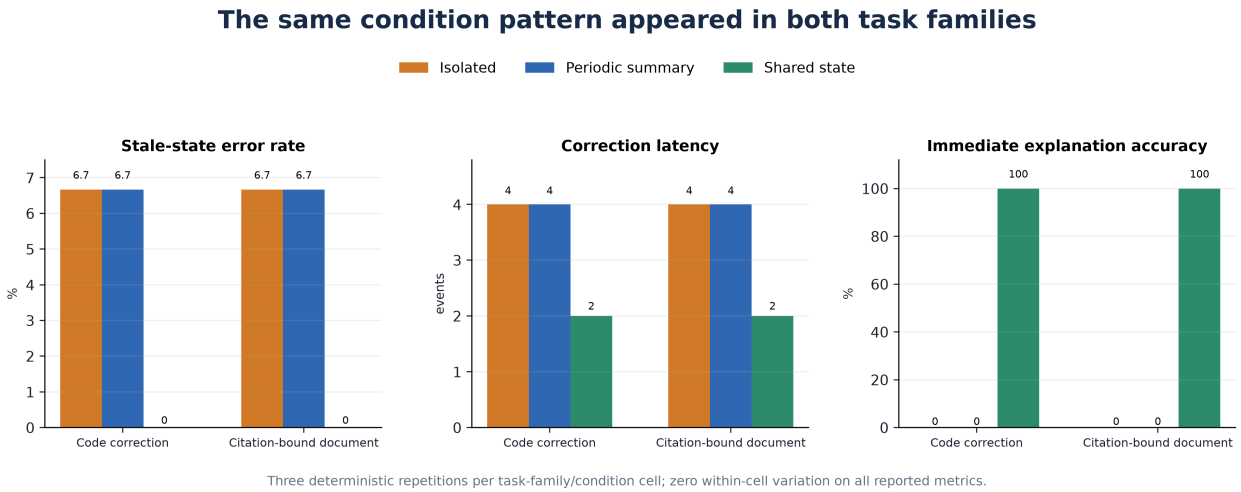

Figure 4. Replication across the two bounded task families. The code-correction and citation-bound document tasks produced the same condition ordering and exact cell means. The zero within-cell variation reflects deterministic pipeline repeatability across three repetitions per task-family/condition cell; it is not a sampling estimate for other tasks, users, or models.

The formal receipt is deliberately narrower than the empirical result. A generic Lean transaction kernel, checked through the serialized compiler lease with exit code zero and no admitted proof, establishes that committed writes require fresh declared versions, execution permission, a valid lease, and a linked source whose declared version is current; every attempted action advances event visibility; blocked actions preserve the file; corrections supersede the old commitment and bind the replacement to the new constraint; and a recovery projection returns current materialized state. An executable checker separately explored 2,048 bounded combinations and audited those invariants against every real run log.

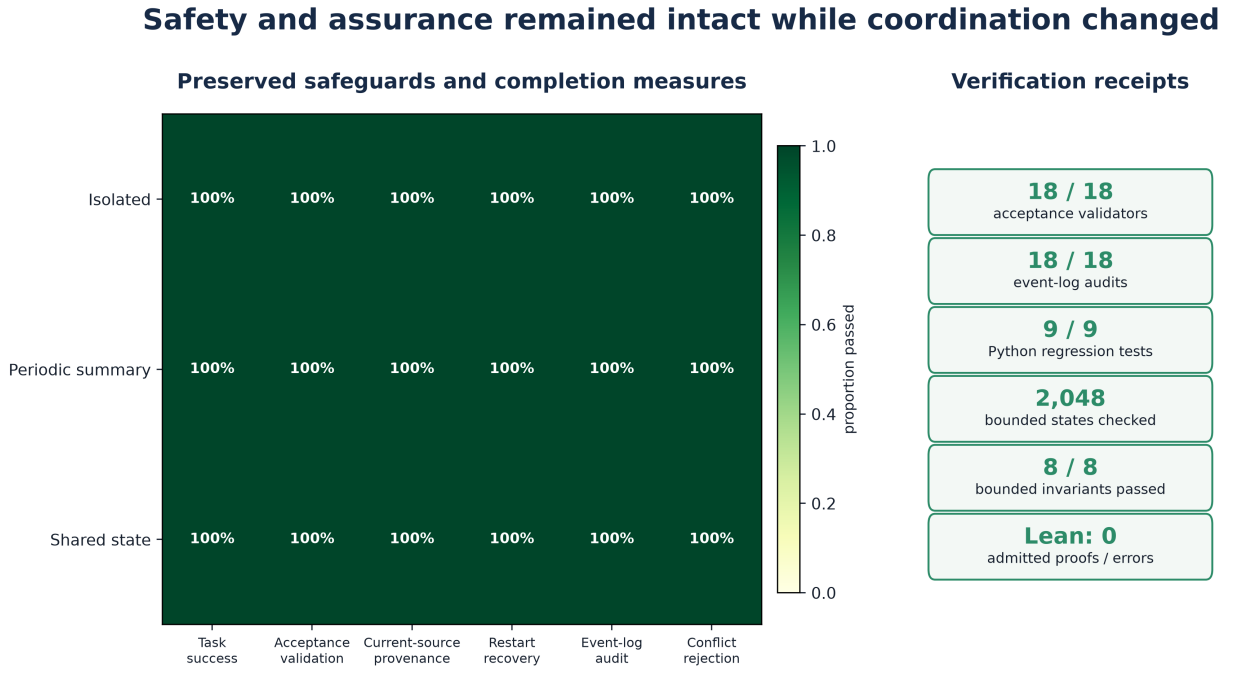

Figure 5. Preserved safeguards and verification. All 18 acceptance validators and all 18 event-log audits passed; all nine Python regression tests passed; the executable checker examined 2,048 bounded states and passed eight of eight stated invariants; and Lean checked the transaction kernel with exit code zero and no admitted proofs. These receipts apply to the released prototype, fixtures, logs, and bounded formal kernel—not to unrestricted multi-agent systems.

The stronger provenance audit found one defect in the preserved code-only branch: a source-manifest update recorded the post-write manifest version as its input source version. The expanded branch now records the version actually declared and read. The earlier package remains preserved for audit, while the corrected 18-run package is the current evidence source.

This result is not yet the full evaluation proposed above. The deterministic adapters encode the tested visibility policies, the repetitions measure pipeline repeatability rather than population uncertainty, and the suite contains only two small task families. The result therefore does not establish general language-model reliability, user preference, human-team performance, or cross-repository generality. Additional task families, held-out instances, model-backed runs, blinded evaluator decisions, trust-calibration data, and ablations remain publication gates for a broader empirical claim.

Safety, governance, and accountability

Functional unity must not erase accountability. Every consequential action remains attributable to a process, model instance, tool call, permission, source state, and owner decision. Required controls include: - explicit user ownership of goals and publication decisions; - capability-scoped permissions and least-privilege role views; - visible active work, pause, cancellation, and rollback controls; - no hidden file mutations or invented background progress; - conflict-safe writes and recoverable versions; - immutable provenance for consequential actions; - separation of private, licensed, and public source material; - bounded ambient behavior with explicit consent; - no sentience claim from persistence, warmth, or operational unity; - documented contribution, review status, accountability, and a distinct decision before journal submission involving any human coauthor.

One project identity must not become one unlimited authority. The permission component P_t in Equation 3 remains process-specific and action-specific. A voice process can know that a deployment is blocked without possessing deployment credentials. A research process can cite a private source internally without authorization to publish it.

Failure conditions and limitations

The architecture does not earn its complexity merely by being internally coherent. It fails as an engineering advantage under any of the following findings:

- summary synchronization performs equally well at lower cost;
- shared state increases conflicting actions or stale-state errors;
- role-conditioned retrieval injects enough irrelevant context to reduce task quality;
- correction propagation is slower or less reliable than explicit point-to-point control;
- provenance storage cannot support useful explanation without unacceptable overhead;
- shared identity causes permission leakage or collapses accountability;
- users over-trust the system because interface unity hides component uncertainty;
- recovery after restart cannot reconstruct the governing commitments and object versions.

The present revision reports a completed deterministic two-task pilot, not the full controlled evaluation. Equations 2–12 are formal contracts, not empirical laws. The first implementation satisfies a bounded subset of

them, but their broader value still depends on model-backed and multi-task results surviving matched baselines and ablations.

Implementation roadmap

A minimal implementation can proceed in seven stages.

- **Object layer.** Define stable identifiers and versions for files, tasks, decisions, sources, people, claims, and results.
- **Event layer.** Record starts, observations, proposals, commits, failures, corrections, reversals, and consolidation events.
- **Commitment layer.** Represent active goals, owner constraints, preconditions, leases, expected consequences, and repair paths.
- **Consequence layer.** Bind predicted effects to returned file, test, tool, user, and external-world evidence; preserve typed residuals and accommodation decisions.
- **Role views.** Implement authorized retrieval for voice, execution, research, reflection, and consolidation.
- **Reciprocal interface.** Make exact execution state discussable through voice and make conversational corrections actionable during execution.
- **Benchmark harness.** Run isolated, summary-synchronized, and shared-state conditions with matched models, tasks, budgets, and blinded scoring.

The implementation should start with one repository and one document project before expanding to multiple repositories, long-running missions, ambient interaction, or multi-user collaboration.

Discussion

Multi-agent systems are often described by counting models or assigning personas. The more useful question is whether the processes participate in one current causal organization. A system of five agents can be operationally fragmented, while one model occupying five roles across time can remain operationally unified if object identity, version, commitments, events, correction, and provenance are conserved and action consequences return to revise the same state.

This reframes memory. A long transcript is not automatically a persistent project mind, and a database is not automatically shared understanding. The relevant property is whether stored state changes the right process at the right time while preserving the ability to explain why. The architecture therefore combines memory with routing, transactions, permissions, and correction.

It also reframes interruption. A new user message is not merely appended text. It can be a causal event that supersedes commitments, stops work, changes the accepted artifact, or requires a new test. Treating interruption as governed state transition is necessary if a conversational interface is to remain genuinely coupled to execution.

Finally, the architecture offers a disciplined relationship between artificial systems and OCA. Both can be described as bounded stateful agents acting through channels and changing future routing through consequences. The comparison becomes scientifically useful only when the physical and mathematical differences remain visible. UAW supplies an artificial implementation target; it does not validate NAPOT, neural oscillation claims, or consciousness.

Conclusion

The paper addresses a specific problem: specialized AI agents often share descriptions of a project without sharing its causally current state. The proposed solution gives persistent objects stable identities and versions; conditions actions on current permissions and preconditions; exposes events, failures, and corrections; binds intended effects to returned consequences; and allows role-specific processes to read, act through, and revise one governed project state. After that operation is understood, it is named the Unified Agentic Workspace.

The conceptual advance is a project-level construction synthesized from the prior source chain. OCA supplies bounded agents and compositional unity; *Building Sentient Beings* supplies one mind with multiple bodies and a shared Totality; *Embodied Neural Rendering* supplies the closed prediction–action–return–update sequence; GIL and μ-GIL supply shared memory, active perception, simulation, assimilation, accommodation, and auditable action selection; SAN v2.2 and the Systems Whitepaper supply the Artificial Neurology translation and the direct shared-agency prediction. UAW makes that joined architecture executable as one project intelligence with multiple permission-bounded hands.

The central claim is testable. In the deterministic two-task pilot, shared state reduced the fixture's stale-state error and correction latency, improved immediate state-explanation accuracy, and maintained task success, provenance, conflict safety, and recovery. This is evidence that the transaction mechanism works as designed, not yet evidence that model-backed agents improve across tasks. If matched summary synchronization performs equally well in the broader evaluation, or if shared state creates greater conflict and leakage, the architecture must be revised or rejected.

The resulting system can be described as one intelligence with multiple hands only in the operational sense established by the unity invariants. Consciousness and sentience remain separate questions. This bounded claim is strong enough to support an implementation program and narrow enough to be falsified.

Appendix A: Implemented visual and numerical evidence

- **Figure 1, system context:** user authority, the UAW boundary, external evidence, and project/tool surfaces.
- **Figure 2, component architecture:** role interfaces, authorized views, transactions, versioned state, corrections, leases, event provenance, adapters, and recovery.
- **Figure 3, condition outcomes:** stale-state error, correction latency, immediate explanation accuracy, and user relay burden.
- **Figure 4, task-family replication:** the same measured condition pattern in code-correction and citation-bound document tasks.
- **Figure 5, safeguards and verification:** completion/safety measures plus validator, audit, test, bounded-checker, and Lean receipts.
- **Expanded measures:** machine-readable condition means, exact contrasts, task-family splits, within-cell repeatability, source hashes, and claim boundary in `data/expanded-measures.json` and `data/expanded-measures.csv`.

Every chart is generated from the sealed `outcomes.csv` and reproducibility receipt. The visual manifest records source and output hashes. No chart presents deterministic repetitions as population samples or confidence evidence.

Appendix B: Authorship, acknowledgment, and preprint scope

Micah Blumberg is the sole author of this revision and is responsible for the present synthesis, formal shared-context model, unity and transaction invariants, voice–execution reciprocity, consequence-reconciliation

model, evaluation design, reference application, evidence package, figures, and manuscript. Michael S. P. Miller is acknowledged for review feedback emphasizing a working prototype, expanded measured results, and system-context and component-architecture diagrams. His relevant prior work on the Piagetian Modeler/GIL lineage, *Building Sentient Beings*, and μ-GIL is cited in the ordinary scholarly record. This acknowledgment does not assign authorship, endorsement, or responsibility for the claims in this paper.

The preprint is intentionally narrower than a general empirical claim. The released materials support the deterministic pilot described here; held-out task families, model-backed runs, blinded evaluation, trust-calibration data, runtime and resource measurements, and mechanism ablations are the next evidence program.

Appendix C: Materials reviewed for this draft

- Blumberg's May 2025 OCA First Draft extraction, preserved locally and hash-bound.
- Blumberg and Miller's 2025 *Building Sentient Beings*.
- Blumberg's *Embodied Neural Rendering in Self-Aware Networks*.
- The 2026 μ-GIL BICA manuscript.
- Blumberg's published *Self-Aware Networks* v2.2.
- Blumberg's 2026 *Self-Aware Networks Systems Whitepaper*.
- Blumberg's 2026 machine-consciousness measurement protocol for the separate consciousness boundary.

References

1. Blumberg, M. (2025). *Self Aware Networks: Oscillatory Computational Agency*. First-draft source preserved in the Self Aware Networks research corpus.
2. Blumberg, M. (2026). *Self-Aware Networks: A Falsifiable Theory-and-Methods Framework for Phase-Gradient Routing and Recurrent Oscillatory Reconstruction*, version 2.2. Self Aware Networks Research Institute. https://zenodo.org/records/16922401
3. Blumberg, M., and Miller, M. S. P. (2025). *Building Sentient Beings*. https://zenodo.org/records/16930405
4. Blumberg, M. (2026). *Embodied Neural Rendering in Self-Aware Networks: A Falsifiable Model of Vision, Sensorimotor Feedback, Body Maps, and the Gamma Consideration Sandwich*. https://zenodo.org/records/21331180
5. Mattey, M., Miller, M. S. P., Dison, D. R., Leveille, T., Miller, M., and Blumberg, M. (2026). *mu-GIL: A Spatial Abstraction Learner*. BICA 2026 manuscript.
6. Blumberg, M. (2026). *Self-Aware Networks: A Systems Whitepaper on Distributed Neural Rendering, Memory Scaling, and Embodied Action*. Self Aware Networks Research Institute final preprint.
7. Blumberg, M. (2026). *Measuring Candidate Machine-Consciousness Mechanisms in Self-Aware Networks: A TCNR Claim-Vector Battery with Adversarial Controls, Perturbation, Rescue, and Abstention*. Self Aware Networks Research Institute final preprint.